

Universidad Tecnológica Nacional

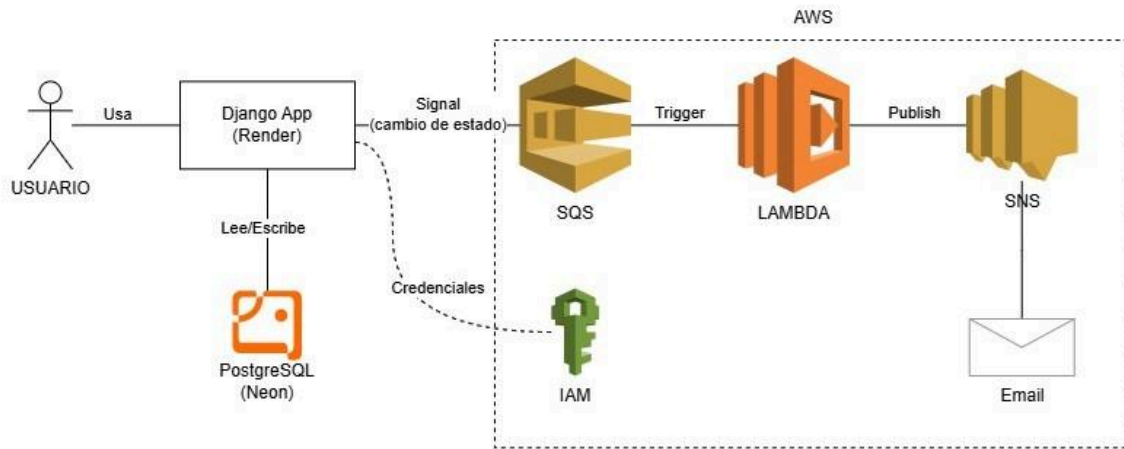
Desarrollo de Software Cloud

TicketLite Cloud

Entrega 1

Muñoz Joaquin

1. Diagrama de arquitectura



El diagrama representa el flujo completo del sistema: la aplicación Django, desplegada en Render, gestiona la lógica de negocio y persiste los datos en PostgreSQL (Neon). Cuando el estado de un ticket cambia, un signal de Django publica un mensaje en una cola Amazon SQS usando las credenciales del usuario IAM creado para este proyecto. La cola dispara automáticamente una función AWS Lambda, que procesa el evento y publica la notificación a través de Amazon SNS / SES, la cual llega como email a un canal centralizado de notificaciones. La ejecución de la Lambda queda registrada en Amazon CloudWatch para su monitoreo.

2. Justificación de los componentes

Componente	Justificación
Django + Render	Backend y frontend integrados (templates server-side). Se descartó separar frontend/backend porque, para el alcance de un sistema de tickets interno de un equipo chico, la complejidad adicional de mantener dos aplicaciones no se traduce en un beneficio real. Render permite un deploy simple.
PostgreSQL (Neon)	Base de datos relacional gestionada. Se eligió por sobre una base autoadministrada para minimizar la carga operativa (backups, parches, disponibilidad).
Amazon SQS	Cola de mensajes que desacopla el evento de negocio (cambio de estado de un ticket) del procesamiento de la notificación. Permite que la app no dependa de forma síncrona ni bloqueante del envío del aviso, y absorbe picos de carga sin perder mensajes.
Amazon SNS / SES	Servicio gestionado de envío de notificaciones. Evita tener que implementar y mantener lógica propia de envío de emails.
IAM (usuario dedicado)	Usuario con permisos acotados (SQS, Lambda, SNS) en lugar de usar las credenciales root de la cuenta.

3. Setup de infraestructura

La infraestructura fue creada y configurada manualmente en la consola de AWS (cuenta dentro del Free Tier), con los siguientes componentes:

- **Cuenta y billing:** Cuenta de AWS con Free Tier, alerta de presupuesto de gasto cero configurada (aviso automático ante cualquier cargo real).
- **IAM:** Usuario con permisos acotados a los servicios utilizados (SQS, Lambda, SNS), sin uso de credenciales root para el trabajo diario.
- **Amazon SQS:** Cola estándar "ticket-notifications", probada manualmente (envío y recepción de mensajes de prueba).
- **AWS Lambda:** Función "procesar-notificacion-ticket" (Python), con la cola SQS conectada como trigger.
- **Amazon SNS:** Topic "tickets-alerts" con suscripción por email confirmada.
- **Integración con la app:** Signal de Django que publica en la cola ante cada cambio de estado registrado en el historial de un ticket.
- **Configuración:** Variables de entorno para las credenciales de AWS y la URL de la cola, siguiendo el mismo mecanismo ya usado para la configuración de la base de datos.

4. Repositorio y actividad

El desarrollo se realiza sobre un repositorio propio en GitHub, que ya contaba con una base funcional de la aplicación. A partir de este checkpoint, los commits vinculados al desarrollo del TPI van a seguir la convención de Conventional Commits, para mantener trazabilidad clara entre el historial de Git y las decisiones de arquitectura documentadas.

<https://github.com/JoacoM2003/Tickets>